

AI/ML Engineer — Take-Home Screening Assignment

RAG-Based Document Q&A System

Start date	Friday, August 21, 2026
Deadline	11:59 PM IST, Monday, August 24, 2026
Submission	GitHub repo link (preferred), emailed to Shounak.Kanade@atmanconsultants.com
Questions	Reply to this email — we'll respond within one business day

1. Overview

Retrieval-Augmented Generation (RAG) is central to how we ship AI products at **Atman Cloud Consultancy**. This assignment simulates a real feature we build: letting users ask natural-language questions over a set of internal documents (product manuals, policies, technical specs) and get accurate, source-grounded answers.

You will build a small but complete RAG pipeline: PDF ingestion → chunking → embeddings → vector database → retrieval → LLM-generated answer, exposed through a simple API or UI. We care less about a polished frontend and more about how you reason about chunking, retrieval quality, grounding, and failure modes.

2. What You'll Receive

Attached to this email you will find 7 sample PDF documents to use as your knowledge base:

- Product_Manual.pdf — a device/software user guide with numbered sections and a troubleshooting table
- Employee_Handbook.pdf — an HR/company policy document (leave policy, code of conduct, benefits)
- API_Reference.pdf — a technical specification with endpoints, parameters, and code snippets
- FAQ_Support.pdf — a customer-facing FAQ / troubleshooting guide
- Security_Policy.pdf — an information security / data-handling policy with defined terms
- Onboarding_Guide.pdf — a multi-page guide with a table of contents and cross-references
- Pricing_and_SLA.pdf — a shorter doc with pricing tables and SLA terms (tests table extraction)

Note: If for any reason the attachments do not come through, you may substitute any 5–10 public PDFs of similar length and type (10–50 pages each) — just note your substitution in the README.

3. Core Requirements

Your submission must implement the full pipeline below. All 8 items are required; the assignment is not considered complete without them.

- Extract text from all provided PDFs (including handling multi-column layouts or tables reasonably well).
- Implement a sensible chunking strategy (explain your chunk size / overlap choices in the README).
- Generate embeddings for each chunk using an embedding model of your choice.
- Store embeddings in a vector database — FAISS, Chroma, or any other vector DB/store.
- Implement semantic search to retrieve the most relevant chunks for a user's question.
- Use an LLM (any provider — OpenAI, Anthropic, open-source, etc.) to generate a final answer grounded in the retrieved chunks.

- Return the source document name, page number, and chunk for every answer, so a user can verify it.
- Build a simple API (e.g., FastAPI/Flask) or a Streamlit UI where a reviewer can type a question and see the answer + sources.
- Gracefully handle questions whose answer is not present in the documents — the system should say so rather than fabricate an answer.
- Provide a README explaining your architecture, setup steps, and design decisions (see Section 6).

4. Suggested Timeline

Note on timing: your start date and deadline give roughly 24 hours to complete this. That's a tight window for a full RAG pipeline — prioritize the core requirements (Section 3) over bonus items, and if you're at risk of missing the deadline, submit what you have working rather than nothing. The phases below map to the pipeline stages regardless of how much wall-clock time you have.

Phase	Suggested focus	Checkpoint
Phase 1	Project setup, PDF text extraction, chunking strategy, pick embedding model & vector DB.	You can chunk all provided PDFs and store embeddings in a vector DB.
Phase 2	Retrieval pipeline, prompt design, LLM integration, source attribution (doc/page/chunk).	Given a question, you get a grounded answer with citations from a script or notebook.
Phase 3	Build the API or Streamlit UI, handle unanswerable questions, basic error handling.	A working UI/API where a reviewer can ask questions and see sourced answers.
Phase 4	Polish, README, (optional) bonus features, final testing, packaging for submission.	Repo is clean, documented, and reproducible from a fresh environment.

5. README Requirements

Your README is graded as carefully as your code. At minimum, include:

- Architecture diagram or clear step-by-step description of the pipeline (PDF → ... → Answer).
 - Which chunking strategy you used and why (chunk size, overlap, splitting logic).
 - Which embedding model and vector DB you chose and why.
 - How retrieval and prompting work, including how sources are attached to answers.
- Setup & run instructions — someone should be able to clone your repo and run it in under 10 minutes.
- Trade-offs & limitations — what you'd improve with more time, and any known weaknesses.
- A note on API keys — do not commit real keys; use a `.env.example` file.

6. Deliverables & Submission

Item	Details
Source code	Git repository (preferred: GitHub repo shared with us) containing all code.
README.md	Architecture overview, setup & run instructions, design decisions, trade-offs, and limitations.
Demo	A short (3–5 min) screen recording OR a runnable Streamlit/API demo we can start locally with one command.
Sample Q&A log	A file showing 10+ example questions (including at least 2 unanswerable ones) with the system's answers and sources.

Submit by emailing hr@atmancloud.com with the subject line **"AI/ML Assignment — [Your Name]"** containing your repo link or zip file before the deadline above.

8. Evaluation Rubric

We evaluate holistically, but roughly weight categories as follows:

Category	Weight	What we're looking for
Correctness & functionality	25%	PDF ingestion, chunking, embeddings, retrieval, and answer generation all work end-to-end without crashing.
RAG design quality	25%	Sensible chunking strategy, appropriate embedding/vector DB choice, relevant retrieval, grounded answers with citations.
Code quality & structure	15%	Readable, modular, reasonably tested code; sane project structure; use of config/env vars for secrets.
Handling edge cases	15%	Out-of-scope questions are handled gracefully; no confident hallucination when the answer isn't in the documents.
Documentation (README)	10%	Clear architecture diagram/explanation, setup steps, design trade-offs, known limitations.
Bonus / going beyond	10%	Hybrid search, reranking, conversation history, eval metrics, or other thoughtful extras.

9. Ground Rules

- Use of AI coding assistants (Copilot, ChatGPT, Claude, etc.) is allowed and expected — this reflects how we actually work. Please note in the README where you used them.
- Any programming language/framework is fine; Python is most common for this kind of task and is what our team primarily uses.
- Any LLM or embedding provider is fine, including free/local models (e.g., Ollama, sentence-transformers) if you'd prefer not to use paid API credits.

- Do not spend more than the allotted time box — we'd rather see a focused, well-reasoned partial solution than a rushed attempt at everything.
- Plagiarizing a public repo wholesale will be treated as a failed submission — we do check.

10. Questions?

If anything in this assignment is ambiguous, make a reasonable assumption, document it in the README, and proceed — judgment under ambiguity is itself part of what we're evaluating. For logistics questions (deadline extensions, missing attachments, etc.), reply directly to this email.

Good luck — we're looking forward to seeing how you think, not just what you build.